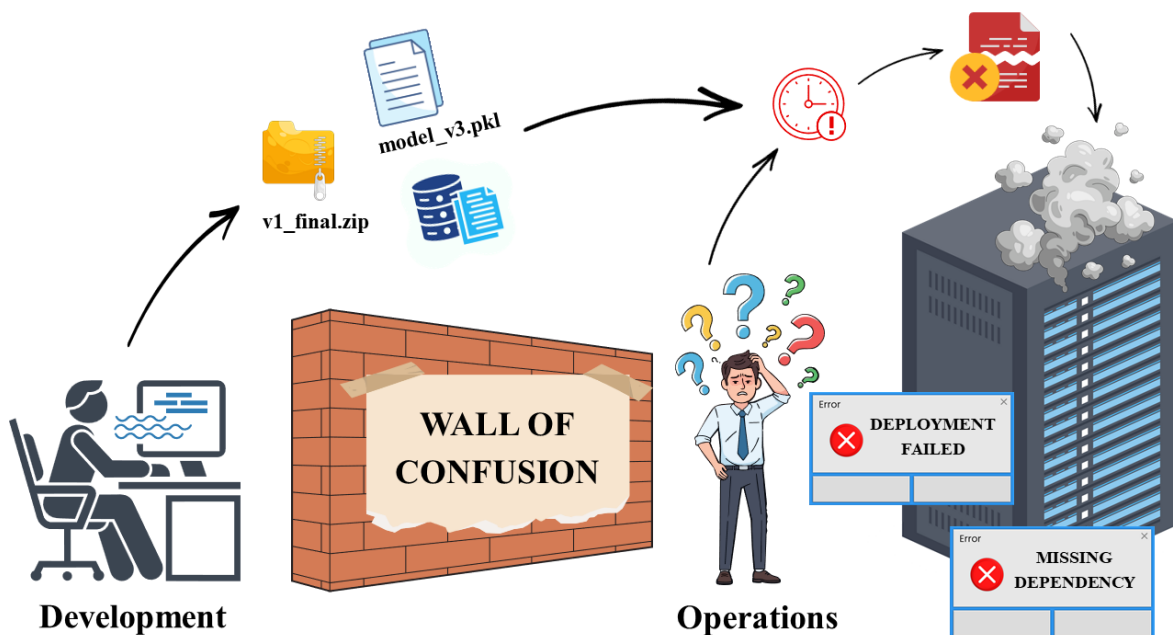


Tutorial: CI/CD với Github Actions, Jenkins và Gitlab CI

Huu-Anh-Duong Ta, Dang-Nha-Nguyen, Duong-Nguyen-Thuan

I. Bối cảnh

Để hiểu rõ vai trò thiết yếu của CI/CD, trước tiên chúng ta cần nhìn nhận lại những nút thắt trong mô hình phát triển phần mềm truyền thống, nơi tồn tại một rào cản vô hình nhưng vô cùng nghiêm trọng giữa đội ngũ phát triển (Dev) và đội ngũ vận hành (Ops). Sự rạn nứt mang tính hệ thống này thường được gọi bằng một hình ảnh quen thuộc: “The Wall of Confusion”. Đây là một khái niệm phản ánh sự thiếu đồng bộ và thiếu tin cậy trong quá trình chuyển giao sản phẩm. Một đoạn mã hoạt động ổn định trên môi trường cục bộ hoàn toàn có thể thất bại khi triển khai ra môi trường thực tế, điều đến từ sự khác biệt về cấu hình hệ thống, phiên bản thư viện, môi trường runtime, hoặc thậm chí là quy trình vận hành.



Hình 1: The Wall of Confusion.

Những sai lệch tưởng chừng nhỏ này lại kéo theo hệ quả nghiêm trọng: rủi ro gia tăng, số lần phát sinh lỗi không thể dự đoán, khó khăn trong việc mở rộng hệ thống, và đặc biệt là sự đứt gãy trong chuỗi cung ứng phần mềm. Tất cả tạo nên một bức tường vô hình, nơi Dev và Ops vận hành theo hai quỹ đạo riêng biệt, thiếu sự giao thoa cần thiết để duy trì tính ổn định và linh hoạt của sản phẩm.

Chính vì vậy, việc thiết lập các luồng công việc tự động hóa - CI/CD trở thành một yêu cầu quan trọng nhằm phá bỏ bức tường đó. CI/CD đảm bảo một nguyên tắc cốt lõi: mọi thay đổi đều phải được kiểm thử, xác thực, đóng gói và triển khai trong một tiến trình có thể lặp lại, minh bạch, và không phụ thuộc vào cá nhân. Điều này không chỉ mang lại sự nhất quán về môi trường và cấu hình, mà còn giảm thiểu đáng kể rủi ro do lỗi thủ công, giúp nhóm phát triển phản hồi nhanh hơn với các thay đổi.

Đáng chú ý, trong bối cảnh các hệ thống tích hợp trí tuệ nhân tạo ngày càng trở nên phổ biến, khái niệm CI/CD tiếp tục được mở rộng sang một nhánh đặc thù hơn: MLOps. Nếu như DevOps tập trung vào sự ổn định của mã nguồn và dịch vụ, thì MLOps còn phải đối mặt với những biến số phức tạp hơn như chất lượng dữ liệu, versioning của mô hình, tính tái lập của quy trình huấn luyện, drift trong dữ liệu và mô hình, cũng như tối ưu hóa vòng đời phục vụ mô hình (model serving lifecycle). Bởi vậy, CI/CD trong AI không chỉ kiểm soát code mà còn phải tự động hóa toàn bộ pipeline: từ tiền xử lý dữ liệu, huấn luyện thử nghiệm, đánh giá, cho đến triển khai và giám sát mô hình trong môi trường thực tế.

Dựa trên cơ sở đó, các phần tiếp theo chúng ta sẽ đi sâu phân tích giải pháp thực tế thông qua ba nền tảng CI/CD tiêu chuẩn: Jenkins, GitHub Actions, và GitLab CI.

Mục lục

I.	Bối cảnh	1
II.	Nền tảng lý thuyết	5
II.1.	Continuous Integration & Continuous Delivery	5
II.1.1.	Tổng quan về CI/CD	5
II.1.2.	CI/CD Pipeline	6
II.2.	Các công cụ CI/CD phổ biến	7
II.3.	Jenkins	7
II.3.1.	Kiến trúc: Master-Agent (Controller-Worker)	7
II.3.2.	Core Concepts	7
II.4.	Github Action	8
II.4.1.	Kiến trúc: Event-Driven Workflow	8
II.5.	Gitlab CI	10
II.5.1.	Tổng quan về Gitlab CI/CD	10
III.	Thực hành:	11
III.1.	Triển khai với Jenkins	12
III.1.1.	Bước 1: Xây dựng mô hình và cấu hình ứng dụng với FastAPI	13
III.1.2.	Bước 2: Xây dựng các bài kiểm tra mô hình và API	16
III.1.3.	Bước 3: Cấu hình Dockerfile	19
III.1.4.	Bước 4: Khởi tạo server Jenkins và Xây dựng Jenkinsfile	19
III.1.5.	Bước 5: Thiết lập Docker Credential	23
III.1.6.	Bước 6: Thiết lập các plugins và cấu hình pipeline	24
III.1.7.	Bước 7: Thiết lập Github hook	26
III.1.8.	Bước 8: Thực hiện push code và tiến hành trigger	27
III.2.	Triển khai với Github Actions	29
III.2.1.	Bước 4: Cấu hình CI/CD pipeline và thiết lập Docker Credential	30
III.2.2.	Bước 5: Thực hiện push code và tiến hành trigger luồng CI/CD	33
III.3.	Triển khai với Gitlab CI	34
III.3.1.	Bước 4: Cấu hình .gitlab-ci.yml và thiết lập Docker Credentials	34
III.3.2.	Bước 5: Thực hiện push code và tiến hành trigger luồng CI/CD	38
IV.	Kết luận	39
V.	Tài liệu tham khảo	40
VI.	Source code và Demo	40

Bảng Giải Thích Thuật Ngữ

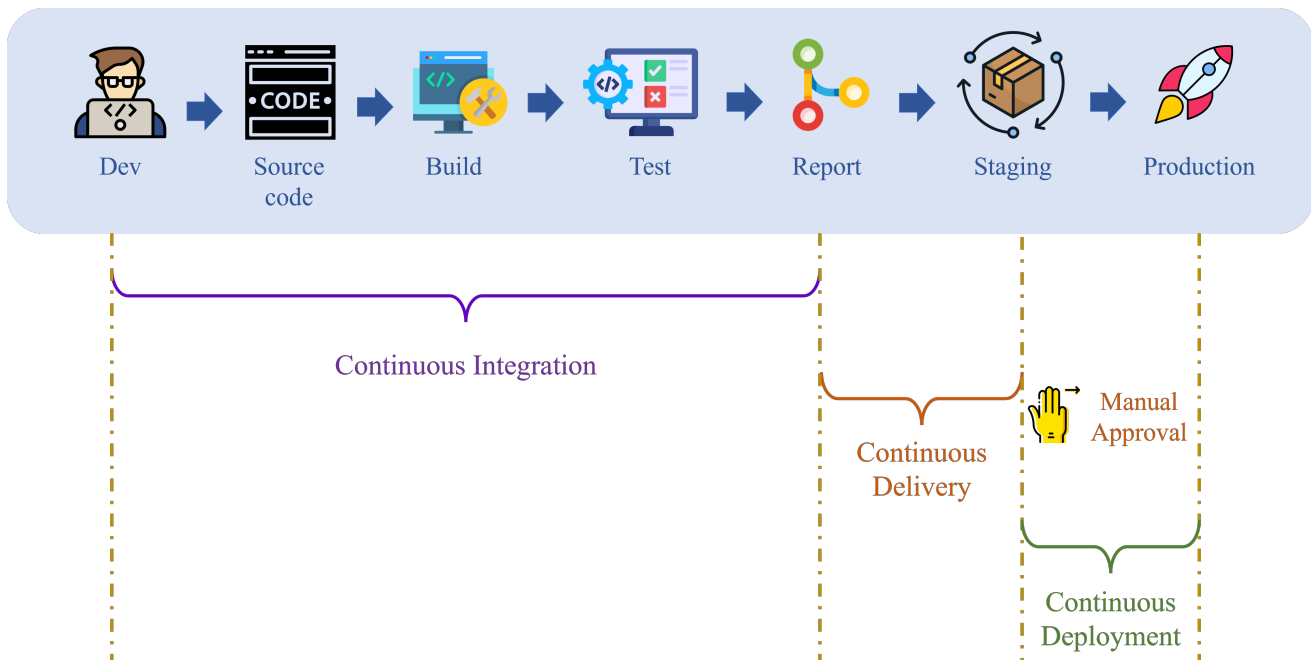
Thuật Ngữ	Giải thích
Development	Môi trường dùng cho việc phát triển, thử nghiệm tính năng mới ở mức độ cá nhân hoặc nhóm nhỏ.
Staging	Môi trường mô phỏng production để kiểm thử toàn diện trước khi triển khai thực tế.
Production	Môi trường vận hành thật, phục vụ người dùng cuối và yêu cầu độ ổn định cao nhất.
Webhook	Cơ chế phản hồi HTTP - Một phương thức cho phép một ứng dụng cung cấp thông tin thời gian thực cho ứng dụng khác.
Artifact	Kết quả được tạo ra sau một bước trong pipeline ví dụ như tệp mô hình (.pkl).
Tunneling	Kỹ thuật thiết lập một kênh truyền dẫn toàn qua Internet đến máy chủ cục bộ (localhost).
Artifact	Sản phẩm trung gian hoặc cuối cùng được tạo ra tại một bước của pipeline, có thể là mô hình đã huấn luyện (.pkl, .onnx), package ứng dụng, file cấu hình hoặc báo cáo. Artifact được lưu trữ để tái sử dụng, kiểm thử, triển khai hoặc tái hiện quy trình.
Orchestration	Cơ chế tự động hóa và điều phối các tác vụ, services và pipeline trong DevOps/MLOps. Orchestration đảm bảo các bước như huấn luyện, kiểm thử, build, deploy và giám sát được thực thi tuần tự hoặc song song theo đúng phụ thuộc. Ví dụ: Kubernetes, Airflow, Argo Workflows.
Polling	Chiến lược kiểm tra định kỳ trạng thái hoặc sự kiện từ một dịch vụ hay nguồn dữ liệu. Trong CI/CD, polling được dùng để phát hiện commit mới, trạng thái job hoặc kết quả huấn luyện khi không có webhook. Nhược điểm: tốn tài nguyên và độ trễ cao hơn so với cơ chế dựa trên sự kiện.
Credentials	Tập thông tin dùng để xác thực và ủy quyền trong hệ thống DevOps/MLOps; bao gồm token, API key, SSH key hoặc thông tin đăng nhập. Credentials dùng để truy cập repository, registry, artifact store, cloud... và cần được bảo vệ bằng secret manager hoặc vault để tránh rò rỉ.

II. Nền tảng lý thuyết

II.1. Continuous Integration & Continuous Delivery

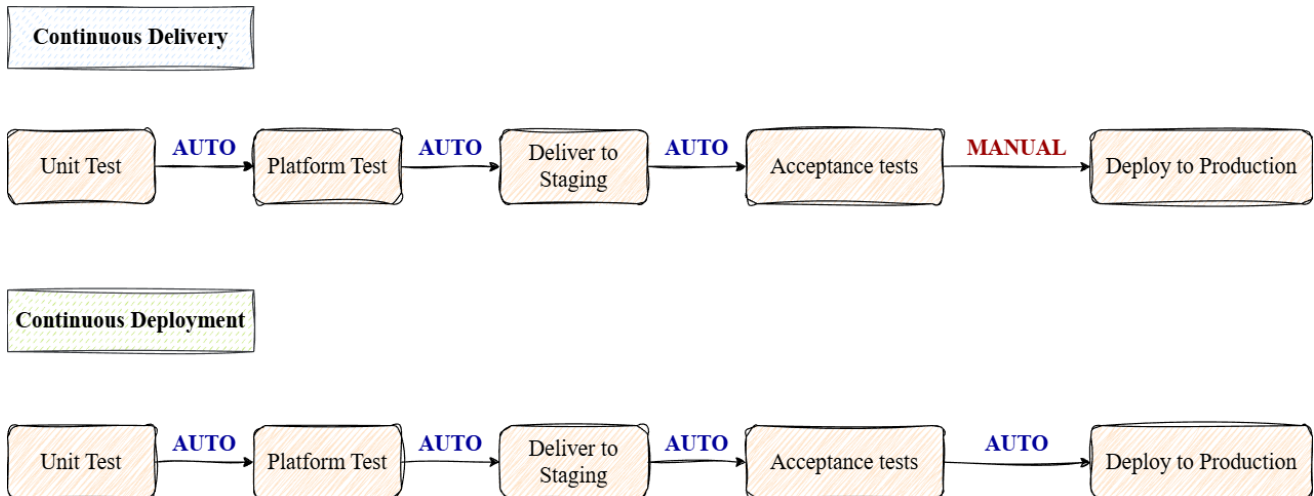
II.1.1. Tổng quan về CI/CD

CI/CD, viết tắt của tích hợp liên tục (continuous integration) và phân phối/triển khai liên tục (continuous deployment), hướng đến việc hợp lý hóa và đẩy nhanh vòng đời phát triển phần mềm:



Hình 2: Tổng quan CI/CD.

- **Continuous integration (CI)** đề cập đến việc tích hợp tự động và thường xuyên các thay đổi mã vào kho lưu trữ mã nguồn dùng chung.
- **Continuous delivery and/or deployment (CD)** là một quy trình mở rộng khái niệm CI bằng cách tự động triển khai mã lên môi trường sản xuất sau khi mã vượt qua tất cả các bài kiểm tra bắt buộc.
- Ngoài ra CD sẽ gồm 2 phần: Continuous delivery dừng lại ở giai đoạn triển khai code lên môi trường staging, và sẽ phải deploy thủ công lên môi trường production, trong khi đó Continuous deployment sẽ tự động triển khai các bản cập nhật vào môi trường production (Hình 3 mô tả rõ sự khác biệt giữa Continuous delivery và Continuous deployment).



Hình 3: Sự khác biệt giữa Continuous Delivery và Continuous Deployment.

II.1.2. CI/CD Pipeline

Sau khi đã có cái nhìn tổng quan về CI/CD chúng ta sẽ đi sâu thêm cách mà một CI/CD pipeline hoạt động, ở đây sẽ có 2 phần chính CI pipeline và CD Pipeline (Hình 3 phía trên) và sẽ tương ứng là các bước chính của các pipeline.

Đầu tiên là phía CI pipeline, với các thành phần cốt lõi sau:

1. **Dev** Giai đoạn lập trình viên phát triển và chỉnh sửa tính năng.
2. **Source Code (Commit)** — Thay đổi được commit lên kho mã nguồn chung, kích hoạt quy trình CI.
3. **Build** Hệ thống tự động biên dịch hoặc đóng gói mã nguồn để kiểm tra tính hợp lệ.
4. **Test** Bao gồm hai lớp kiểm thử:
 - **Unit Tests:** xác minh tính đúng đắn của từng hàm hoặc module nhỏ.
 - **Integration Tests:** kiểm tra sự tương tác giữa các thành phần khi kết hợp với nhau.
5. **Report** Tổng hợp kết quả build và kiểm thử, làm cơ sở đánh giá chất lượng mã nguồn.

Tiếp theo là phía CD pipeline, bao gồm:

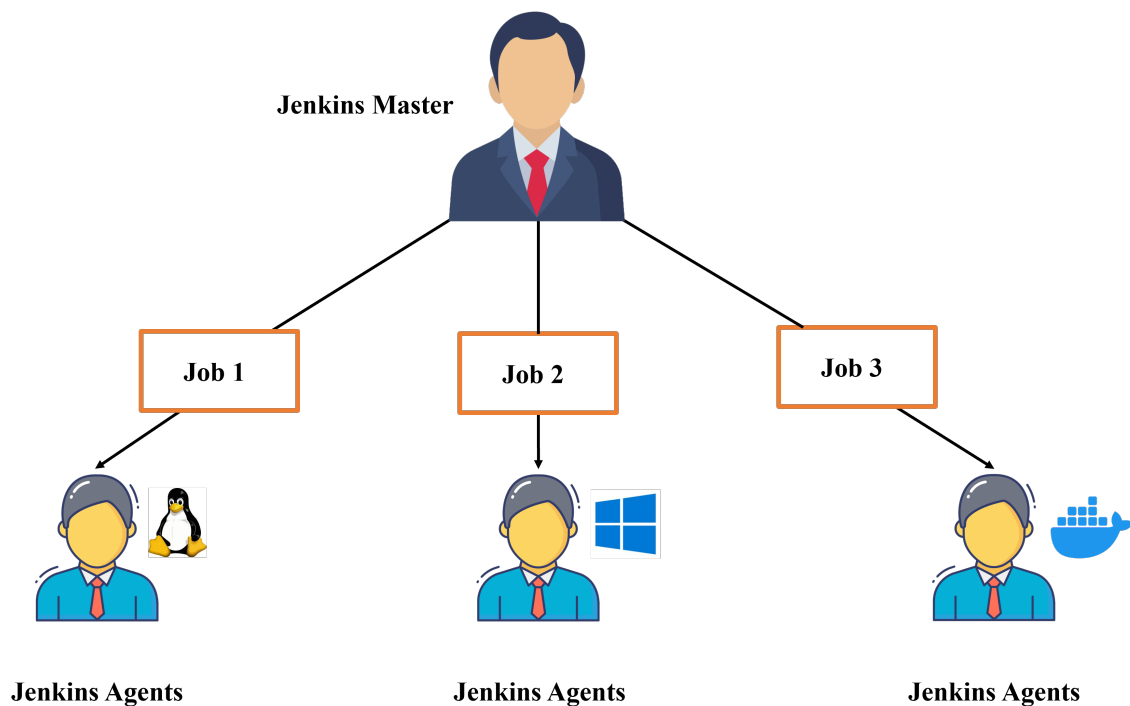
1. **Release (Staging)** Tạo bản phát hành và triển khai lên môi trường staging, mô phỏng điều kiện production để kiểm tra lần cuối.
2. **Deploy (Production)** Triển khai phiên bản đã được xác nhận chất lượng lên môi trường production cho người dùng cuối.

II.2. Các công cụ CI/CD phổ biến

II.3. Jenkins

Jenkins là một máy chủ tự động hóa mã nguồn mở, viết bằng Java, giúp tự động hóa các quy trình trong phát triển phần mềm như tích hợp liên tục (CI) và phân phối liên tục (CD).

II.3.1. Kiến trúc: Master-Agent (Controller-Worker)



Hình 4: Kiến trúc Master-Agent của Jenkins

Jenkins hoạt động dựa trên kiến trúc phân tán (Distributed Build Architecture) được trực quan với hình số 4. Đây là điểm cốt yếu khi áp dụng cho MLOps, nơi yêu cầu tài nguyên tính toán lớn.

- **Jenkins Master (Controller):** Đóng vai trò là "bộ não" trung tâm. Nhiệm vụ của nó là lưu trữ cấu hình, điều phối lịch trình (scheduling), giám sát các Job và phân phối tải. Master thường không trực tiếp thực thi các tác vụ nặng.
- **Jenkins Agent (Node/Slave):** Là các máy trạm thực thi. Trong bối cảnh MLOps, Agent thường là các máy chủ chứa GPU (ví dụ: EC2 instance có NVIDIA Tesla hoặc máy local có GPU). Master sẽ gửi lệnh xuống Agent để thực hiện việc training model.

II.3.2. Core Concepts

Ở đây sẽ có 2 concepts chính là Pipeline và Stage & Step

- Pipeline as Code: Thay vì cấu hình thủ công trên giao diện UI, toàn bộ quy trình CI/CD được định nghĩa bằng code trong một file text tên là Jenkinsfile. File này sử dụng ngôn ngữ Groovy DSL (Domain Specific Language). Có hai cú pháp chính: Scripted (cổ điển, linh hoạt) và Declarative (hiện đại, dễ đọc, được khuyến nghị).
- Như đã đề cập ở trên Pipeline là tập hợp toàn bộ quy trình và trong đó có các Stages là các khối logic lớn (ví dụ như "Build", "Test", "Deploy") và trong các Stages là các Steps - đơn vụ thực thi nhỏ nhất (ví dụ như: lệnh shell, lệnh git).

Dưới đây là code ví dụ cấu trúc Declarative Pipeline:

```

Jenkinsfile
1  pipeline {
2    agent { label 'gpu-node' } // Specify to run on Agent labeled 'gpu-node'
3
4    stages {
5      stage('Data Preparation') {
6        steps {
7          sh 'dvc pull'
8        }
9      }
10     stage('Model Training') {
11       steps {
12         sh 'python train.py'
13       }
14     }
15   }
16 }

```

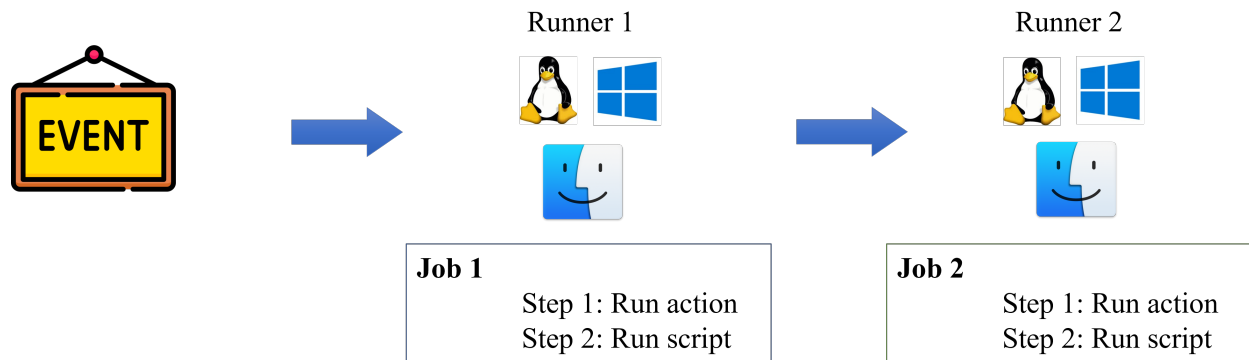
Đoạn code trên định nghĩa một quy trình chạy trên node chuyên dụng (gpu-node). Nó được chia làm 2 giai đoạn (stages) tuần tự: chuẩn bị dữ liệu và huấn luyện mô hình. Lệnh sh được dùng để gọi các script hệ thống.

II.4. Github Action

GitHub Actions là một công cụ tự động hóa mạnh mẽ được cung cấp bởi GitHub, cho phép các nhà phát triển tự động hóa các tác vụ liên quan đến quy trình phát triển phần mềm của họ. Nó cho phép tạo ra các quy trình làm việc (workflows) tùy chỉnh, có thể được kích hoạt bởi nhiều sự kiện khác nhau, chẳng hạn như khi đẩy code (push) lên kho lưu trữ, tạo yêu cầu kéo (pull request) hoặc các sự kiện được lên lịch sẵn. Các quy trình công việc này có thể được sử dụng cho nhiều mục đích khác nhau, bao gồm Tích hợp liên tục (CI), Triển khai liên tục (CD), kiểm thử, và nhiều hơn nữa.

II.4.1. Kiến trúc: Event-Driven Workflow

Khác với mô hình Master-Agent cứng nhắc, GitHub Actions hoạt động dựa trên Events.



Hình 5: Kiến trúc Event-Driven Workflow của Github Actions

Ở đây sẽ có một số concepts cần phải lưu ý như sau:

- **Workflows:** Một quy trình tự động hóa có thể cấu hình, được định nghĩa trong thư mục `.github/workflows/`. Một repo có thể có nhiều workflows cho các mục đích khác nhau (Test, Deploy, Auto-labeling).
- **Event:** Một sự kiện xảy ra trên kho lưu trữ (Repo) (Push, Pull Request, Release, hoặc Schedule CRON) dùng để kích hoạt workflow.
- **Actions:** Action là một ứng dụng nhỏ, đóng gói sẵn logic phức tạp. Thay vì viết script bash dài dòng để cài đặt Python, login Docker Hub, hay setup AWS CLI, ta chỉ cần gọi một Action có sẵn (uses: action-name). **Note:** Người dùng có thể tìm thấy các actions để sử dụng trong workflow của mình trên GitHub Marketplace.
- **Runners:** Máy ảo (do GitHub host hoặc người dùng tự host) nhận Workflow để xử lý.
- **Jobs:** Một Job là tập hợp các Steps được thực thi trên cùng một Runner, mặc định các Jobs chạy song song (Parallel) hoàn toàn độc lập để tối ưu thời gian nên để thiết lập chuỗi phụ thuộc (Job B chỉ chạy khi Job A thành công) thì cần từ khóa "needs".
- **Steps:** Đơn vị nhỏ nhất trong Job. Nó có thể là một lệnh Shell hoặc một Action. Vì các Steps trong cùng một Job chạy trên cùng một máy ảo (Runner), chúng có thể chia sẻ dữ liệu qua Filesystem hoặc Environment Variables. Ví dụ: Step 1 build model, Step 2 test model đó (không cần upload/download artifact giữa 2 bước này).

Dưới đây là một ví dụ code về workflow cơ bản sử dụng Github Action

example.yml

```

1 name: MLOps Workflow
2 on: [push]
3
4 jobs:
5   training:
6     runs-on: self-hosted # Runs on a dedicated GPU
7     steps:
8       - uses: actions/checkout@v3 # Standard action to get code
9

```

```
10     - name: Setup Python
11       uses: actions/setup-python@v4 # Standard action to install Python
12       with:
13         python-version: '3.9'
14
15     - name: Run Training Script
16       run: python train.py
```

Workflow kích hoạt khi có lệnh push. Job training được chỉ định chạy trên self-hosted runner (máy GPU cá nhân) và sử dụng 2 Actions có sẵn từ Marketplace (checkout và setup-python) giúp code gọn gàng hơn nhiều so với việc viết lệnh shell thủ công, sau đó mới chạy lệnh training.

II.5. Gitlab CI

GitLab CI là một công cụ phát triển phần mềm trong hệ sinh thái GitLab cho phép thực hiện các phương pháp tích hợp liên tục (CI) và triển khai liên tục (CD) ngay trên kho mã nguồn.

II.5.1. Tổng quan về Gitlab CI/CD

Đầu tiên về nguyên lý lý hoạt động: Sức mạnh của GitLab nằm ở file cấu hình .gitlab-ci.yml. Toàn bộ quy trình tự động hóa được định nghĩa dưới dạng mã (Pipeline-as-Code) và nằm ngay trong kho mã nguồn (Repository) của người phát triển. Ví dụ: ngay khi bạn đẩy code (push) chứa file này lên repository, GitLab sẽ tự động phát hiện và kích hoạt một Pipeline tương ứng. Để làm chủ GitLab CI/CD, bạn cần nắm vững cấu trúc phân cấp sau:

- **GitLab Pipeline:** Trong GitLab CI/CD, pipeline là các thành phần cấp cao nhất giúp tích hợp, triển khai và phân phối liên tục. Pipeline bao gồm các jobs để xác định chức năng của pipeline (ví dụ: compiling or testing code) và các stages xác định thời điểm pipeline nên chạy một tác vụ (ví dụ: chạy kiểm thử sau khi biên dịch mã).
- **Stages:** Pipeline được chia nhỏ thành các giai đoạn tuần tự. Đây là ranh giới logic để quản lý quy trình. (Ví dụ: Build -> Test -> Deploy)
- **Jobs:** Đây là đơn vị thực thi nhỏ nhất, là nơi nhà phát triển viết các câu lệnh script (như npm install, python train.py).
- **GitLab Server:** Quản lý kho code, quản lý pipeline và điều phối job.
- **GitLab Runner:** Một ứng dụng mã nguồn mở hoạt động độc lập, có nhiệm vụ nhận job từ Server và thực thi nó.
- **Artifacts:** Lưu trữ kết quả đầu ra và chuyển giao giữa các Stages (Ví dụ như dùng để lưu các file Model sau khi train xong).
- **CI/CD Variables:** Trong MLOps, chúng ta thường cần truy cập vào Cloud Storage (AWS S3, Google Drive) để lấy dữ liệu lớn. GitLab Variables cho phép lưu trữ các thông tin nhạy cảm này (API Key, Credentials) một cách an toàn và inject chúng vào môi trường chạy

Job mà không cần hard-code trong file `.gitlab-ci.yml`. Dưới đây là một code ví dụ cấu trúc `.gitlab-ci.yml`

```
.gitlab-ci.yml

1 stages:
2   - build
3   - train
4
5 install_dependencies:
6   stage: build
7   script:
8     - pip install -r requirements.txt
9
10 train_model:
11   stage: train
12   script:
13     - python train.py
14   artifacts:
15     paths:
16       - model.pkl
17       - metrics.json
```

Pipeline trên có 2 stages. Job `train_model` thuộc stage `train`. Sau khi chạy script python, nó sẽ gom file `model.pkl` và `metrics.json` đóng gói lại thành Artifact và gửi lên GitLab Server để lưu trữ.

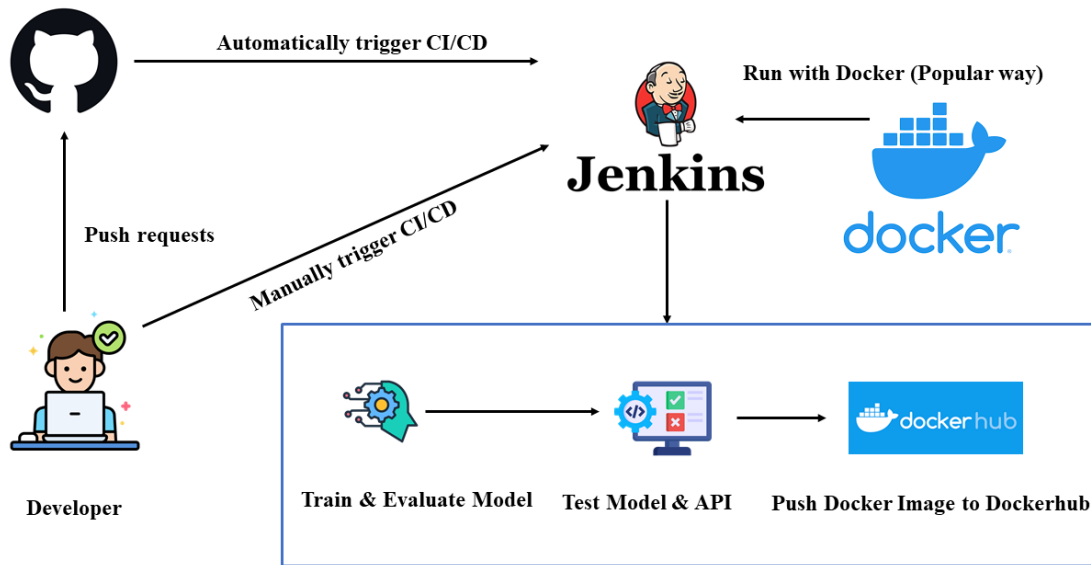
III. Thực hành:

Trong phần thực hành này, chúng ta sẽ huấn luyện và đánh giá một mô hình ML đơn giản (Random Forest Classifier) từ scikit-learn trên bộ Iris dataset. Đây là bộ dữ liệu cổ điển trong machine learning, bao gồm 150 mẫu với 4 đặc trưng (chiều dài và chiều rộng của đài hoa, cánh hoa) để phân loại 3 loài hoa Iris (Setosa, Versicolor, Virginica). Ngoài ra sẽ xây dựng API dự đoán sử dụng FastAPI. Mục tiêu bài toán:

- **Xây dựng ứng dụng ML Production-ready:** Tạo một REST API sử dụng FastAPI để serve model predictions, cho phép người dùng gửi thông số hoa và nhận về kết quả phân loại với độ tin cậy.
- **Tự động hóa quy trình phát triển:** Thiết lập một pipeline CI/CD hoàn chỉnh sử dụng Jenkins/Github Action/ Gitlab CI để tự động hóa toàn bộ quy trình từ khi nhà phát triển push request cho đến khi ứng dụng được đóng gói và triển khai.
- **Containerization và Distribution:** Đóng gói ứng dụng cùng với model đã được train sẵn vào Docker image, sau đó đẩy lên Docker Hub để dễ dàng triển khai trên bất kỳ môi trường nào.

III.1. Triển khai với Jenkins

Với việc triển khai với Jenkins chúng ta sẽ có sơ đồ toàn cảnh như sau:

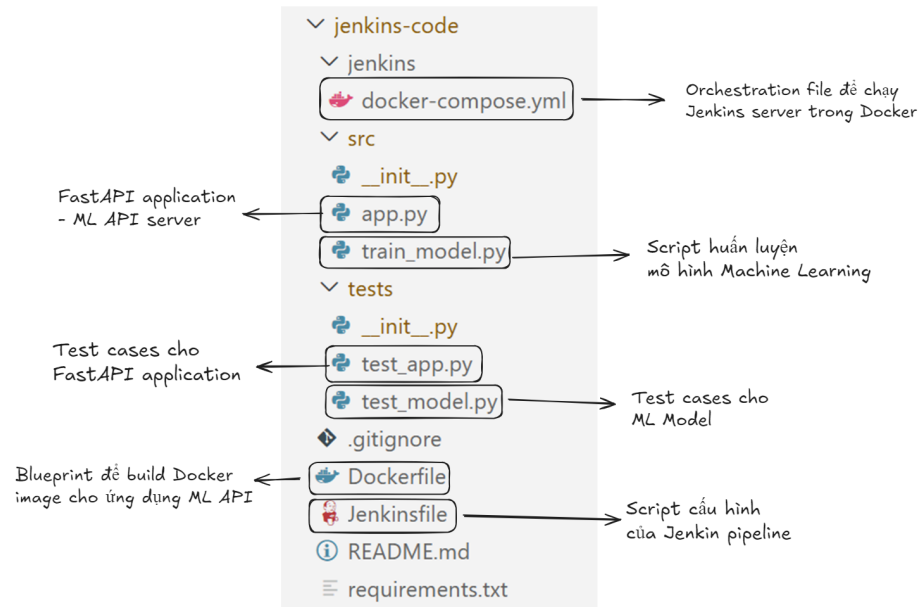


Hình 6: Cấu trúc bài thực hành với Jenkins

Phân tích workflow:

- Developer phát triển và cải tiến model hoặc API code
- Push code lên GitHub repository
- GitHub webhook tự động trigger Jenkins pipeline
- Jenkins thực hiện: Huấn luyện lại model với dữ liệu mới, tiếp theo chạy suite tests để đảm bảo chất lượng (unit tests cho model và integration tests cho API), cuối cùng build Docker image chứa model đã train và API sau đó push image lên Docker Hub với versioning.

Tiếp theo chúng ta sẽ cấu trúc code để hiểu rõ hơn về cách triển khai:



Hình 7: Cấu trúc bài thực hành với Jenkins.

III.1.1. Bước 1: Xây dựng mô hình và cấu hình ứng dụng với FastAPI

train_model.py

```

1 import pickle
2 from sklearn.datasets import load_iris
3 from sklearn.model_selection import train_test_split
4 from sklearn.ensemble import RandomForestClassifier
5 from sklearn.metrics import accuracy_score
6 import os
7
8 def train_and_save_model():
9     # Load dataset
10    iris = load_iris()
11    X, y = iris.data, iris.target
12
13    # Split data
14    X_train, X_test, y_train, y_test = train_test_split(
15        X, y, test_size=0.2, random_state=42
16    )
17
18    # Train model
19    model = RandomForestClassifier(n_estimators=100, random_state=42)
20    model.fit(X_train, y_train)
21
22    # Evaluate
23    y_pred = model.predict(X_test)
24    accuracy = accuracy_score(y_test, y_pred)
25    print(f"Model accuracy: {accuracy:.2f}")
26
27    # Save model

```

```

28     os.makedirs("models", exist_ok=True)
29     with open("models/iris_model.pkl", "wb") as f:
30         pickle.dump(model, f)
31
32     print("Model saved to models/iris_model.pkl")
33     return accuracy
34
35 if __name__ == "__main__":
36     train_and_save_model()

```

Dựa trên kiến trúc mô hình và bộ dữ liệu đã đề xuất, quy trình được tiến hành theo trình tự năm bước trong file `train_model.py`: tải dữ liệu, phân chia tập dữ liệu (data partitioning), huấn luyện (training), đánh giá hiệu năng (evaluation) và cuối cùng là lưu trữ trọng số mô hình (model saving)

app.py

```

1
2 from fastapi import FastAPI, HTTPException
3 from pydantic import BaseModel
4 import pickle
5 import numpy as np
6 from typing import List
7 import os
8
9 app = FastAPI(title="Iris Classification API", version="1.0.0")
10
11 # Load model
12 MODEL_PATH = os.path.join(os.path.dirname(__file__), "models", "iris_model.pkl")
13 model = None
14
15 class PredictionInput(BaseModel):
16     sepal_length: float
17     sepal_width: float
18     petal_length: float
19     petal_width: float
20
21 class PredictionOutput(BaseModel):
22     prediction: int
23     class_name: str
24     confidence: float
25
26 def load_model():
27     global model
28     if not os.path.exists(MODEL_PATH):
29         raise FileNotFoundError(f"Model file not found at {MODEL_PATH}")
30
31     with open(MODEL_PATH, "rb") as f:
32         model = pickle.load(f)
33     print("Model loaded successfully")
34
35 @app.on_event("startup")

```

```

36 async def startup_event():
37     load_model()
38
39 @app.get("/")
40 async def root():
41     return {
42         "message": "Iris Classification API",
43         "status": "running",
44         "endpoints": {
45             "health": "/health",
46             "predict": "/predict",
47             "docs": "/docs"
48         }
49     }
50
51 @app.get("/health")
52 async def health():
53     return {
54         "status": "healthy",
55         "model_loaded": model is not None
56     }
57
58 @app.post("/predict", response_model=PredictionOutput)
59 async def predict(input_data: PredictionInput):
60     if model is None:
61         raise HTTPException(status_code=500, detail="Model not loaded")
62
63     # Prepare input
64     features = np.array([
65         input_data.sepal_length,
66         input_data.sepal_width,
67         input_data.petal_length,
68         input_data.petal_width
69     ])
70
71     # Make prediction
72     prediction = model.predict(features)[0]
73     probabilities = model.predict_proba(features)[0]
74     confidence = float(max(probabilities))
75
76     # Class names
77     class_names = ["setosa", "versicolor", "virginica"]
78
79     return PredictionOutput(
80         prediction=int(prediction),
81         class_name=class_names[prediction],
82         confidence=confidence
83     )
84
85 if __name__ == "__main__":
86     import uvicorn
87     uvicorn.run(app, host="0.0.0.0", port=8000)

```

Đoạn mã tập trung triển khai ứng dụng sử dụng thư viện FastAPI. Cơ chế vận hành bao gồm việc tải trước (pre-load) mô hình đã huấn luyện (định dạng .pkl) vào bộ nhớ ngay tại thời điểm khởi chạy. Hệ thống thiết lập các giao diện lập trình ứng dụng (API) thiết yếu: root, /predict (dự đoán) và /health (giám sát trạng thái)

III.1.2. Bước 2: Xây dựng các bài kiểm tra mô hình và API

test_model.py

```

1 import pytest
2 import pickle
3 import os
4 import sys
5 import numpy as np
6 from sklearn.datasets import load_iris
7 from sklearn.ensemble import RandomForestClassifier
8
9 sys.path.insert(0, os.path.join(os.path.dirname(__file__), '..', 'src'))
10 from train_model import train_and_save_model
11
12
13 class TestModelTraining:
14
15     @pytest.fixture(autouse=True)
16     def setup_and_teardown(self):
17         os.makedirs("models", exist_ok=True)
18
19         yield
20
21
22     def test_train_model_success(self):
23         accuracy = train_and_save_model()
24         assert accuracy is not None
25         assert isinstance(accuracy, (float, np.float64))
26         assert 0 <= accuracy <= 1
27
28     def test_model_accuracy_threshold(self):
29         accuracy = train_and_save_model()
30         assert accuracy >= 0.90, f"Model accuracy {accuracy:.2%} is below 90% threshold"
31
32     def test_model_file_created(self):
33         train_and_save_model()
34         assert os.path.exists("models/iris_model.pkl")
35         assert os.path.getsize("models/iris_model.pkl") > 0
36
37     def test_model_can_be_loaded(self):
38         train_and_save_model()
39         with open("models/iris_model.pkl", "rb") as f:
40             model = pickle.load(f)
41         assert isinstance(model, RandomForestClassifier)
42         assert hasattr(model, 'predict')
43         assert hasattr(model, 'predict_proba')

```

```

44
45     def test_loaded_model_can_predict(self):
46         train_and_save_model()
47         with open("models/iris_model.pkl", "rb") as f:
48             model = pickle.load(f)
49             iris = load_iris()
50             X_sample = iris.data[:5]
51
52             predictions = model.predict(X_sample)
53             probabilities = model.predict_proba(X_sample)
54
55             assert len(predictions) == 5
56             assert all(pred in [0, 1, 2] for pred in predictions)
57             assert probabilities.shape == (5, 3)
58             assert all(np.isclose(prob.sum(), 1.0) for prob in probabilities)
59
60     def test_model_parameters(self):
61         train_and_save_model()
62         with open("models/iris_model.pkl", "rb") as f:
63             model = pickle.load(f)
64             assert model.n_estimators == 100
65             assert model.random_state == 42
66             assert model.n_classes_ == 3
67             assert model.n_features_in_ == 4 # Iris has 4 features
68
69 if __name__ == "__main__":
70     pytest.main([__file__, "-v"])

```

Module trên sẽ kiểm tra logic training và đánh giá mô hình. Class TestModelTraining sẽ đảm bảo việc training diễn ra suôn sẻ và tạo ra sản phẩm (artifact) đúng yêu cầu. Ví dụ như hàm test_model_accuracy_threshold kiểm tra mô hình mới train có độ chính xác dưới 90% thì bài test sẽ thất bại, hoặc hàm test_model_file_created kiểm tra xem sau khi chạy hàm train, file "models/iris_model.pkl" có tồn tại hay không, và ngoài ra hàm test_model_parameters sẽ kiểm tra xem mô hình có giữ đúng cấu hình đã định không (ví dụ: n_estimators=100).

test_app.py

```

1 import pytest
2 from fastapi.testclient import TestClient
3 import os
4 import sys
5
6 sys.path.insert(0, os.path.join(os.path.dirname(__file__), '..', 'src'))
7 from app import app, load_model
8
9
10 load_model()
11
12 client = TestClient(app)
13
14 def test_root():

```

```
15 response = client.get("/")
16 assert response.status_code == 200
17 assert "message" in response.json()
18 assert response.json()["status"] == "running"
19
20 def test_health():
21     response = client.get("/health")
22     assert response.status_code == 200
23     assert response.json()["status"] == "healthy"
24     assert response.json()["model_loaded"] == True
25
26 def test_predict():
27     test_input = {
28         "sepal_length": 5.1,
29         "sepal_width": 3.5,
30         "petal_length": 1.4,
31         "petal_width": 0.2
32     }
33
34     response = client.post("/predict", json=test_input)
35     assert response.status_code == 200
36
37     data = response.json()
38     assert "prediction" in data
39     assert "class_name" in data
40     assert "confidence" in data
41     assert data["class_name"] in ["setosa", "versicolor", "virginica"]
42     assert 0 <= data["confidence"] <= 1
43
44 def test_predict_invalid_input():
45     invalid_input = {
46         "sepal_length": 5.1,
47         "sepal_width": 3.5
48     }
49
50     response = client.post("/predict", json=invalid_input)
51     assert response.status_code == 422 # Validation error
52
53 if __name__ == "__main__":
54     pytest.main([__file__, "-v"])
```

Phần code trên với mục tiêu kiểm tra khi bọc mô hình vào FastAPI, hệ thống có hoạt động như một Web Server hoàn chỉnh không bằng cách sử dụng TestClient của FastAPI để giả lập gửi request mà không cần chạy server thật. Ở đây sẽ bao gồm các bài test như gửi request GET tới /health để kiểm tra xem biến toàn cục model đã được load thành công vào bộ nhớ chưa. Ngoài ra sẽ kiểm tra thêm /predict với hai đầu vào input hợp lệ và không hợp lệ.

III.1.3. Bước 3: Cấu hình Dockerfile

requirements.txt

```
1 fastapi==0.115.0
2 uvicorn==0.32.0
3 scikit-learn==1.5.2
4 numpy==2.1.0
5 pydantic==2.9.0
6 pytest==8.3.0
7 httpx==0.27.0
```

Việc định nghĩa các dependencies cần thiết để cài đặt sẽ thông qua requirements.txt.

Dockerfile

```
1 # Dockerfile for ML API
2 FROM python:3.13-slim
3
4 WORKDIR /app
5
6 # Copy requirements and install dependencies
7 COPY requirements.txt .
8 RUN pip install --no-cache-dir -r requirements.txt
9
10 # Copy application code from src
11 COPY src/train_model.py .
12 COPY src/app.py .
13
14 # Train model
15 RUN python train_model.py
16
17 # Expose port
18 EXPOSE 8000
19
20 # Run application
21 CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

Dockerfile chịu trách nhiệm đóng gói Mô hình ML và API. Từ đó sẽ phục vụ cho bước đẩy Docker Image lên Dockerhub.

III.1.4. Bước 4: Khởi tạo server Jenkins và Xây dựng Jenkinsfile

docker-compose.yml

```
1 version: '3.8'
2
3 services:
4   jenkins:
5     image: jenkins/jenkins:lts
```

```

6   container_name: jenkins-server
7   privileged: true
8   user: root
9   ports:
10  - "8080:8080"
11  - "50000:50000"
12  volumes:
13  - jenkins_home:/var/jenkins_home
14  - /var/run/docker.sock:/var/run/docker.sock
15  networks:
16  - jenkins-network
17  restart: unless-stopped
18  entrypoint: /bin/bash -c "
19      apt-get update &&
20      apt-get install -y python3 python3-pip python3-venv docker.io curl &&
21      usermod -aG docker jenkins &&
22      chmod 666 /var/run/docker.sock &&
23      /usr/local/bin/jenkins.sh
24  "
25
26 volumes:
27   jenkins_home:
28     driver: local
29
30 networks:
31   jenkins-network:
32     driver: bridge

```

File docker-compose.yml chịu trách nhiệm dựng lên Hệ thống Jenkins (CI/CD Server). Ở đây sẽ có một số điểm đặc biệt như sau:

- **volumes:** `/var/run/docker.sock:/var/run/docker.sock`. Dòng này cho phép Jenkins (chạy trong container) có thể gọi lệnh Docker của máy chủ (Host) để build image cho ứng dụng. Đây là kỹ thuật Docker-in-Docker.
- **entrypoint:** Một đoạn script cài đặt thủ công python3, pip, venv và docker vào bên trong container Jenkins ngay khi khởi động. Điều này giúp Jenkins có đủ công cụ để chạy các lệnh trong Pipeline.

Jenkinsfile

```

1 pipeline {
2   agent any
3
4   environment {
5     DOCKER_IMAGE = "your-dockerhub-username/iris-ml-api"
6     DOCKER_TAG = "${BUILD_NUMBER}"
7     DOCKER_CREDENTIALS_ID = "dockerhub-credentials"
8   }
9
10  stages {
11    stage('Checkout') {

```



```
12     steps {
13         echo 'Checking out code from GitHub...'
14         checkout scm
15     }
16 }
17
18 stage('Setup Python Environment') {
19     steps {
20         echo 'Setting up Python environment...'
21         sh '''
22             python3 -m venv venv
23             . venv/bin/activate
24             pip install --upgrade pip
25             pip install -r requirements.txt
26         '''
27     }
28 }
29
30 stage('Train Model') {
31     steps {
32         echo 'Training ML model...'
33         sh '''
34             . venv/bin/activate
35             cd src
36             python train_model.py
37             cd ..
38         '''
39     }
40 }
41
42 stage('Test Model') {
43     steps {
44         echo 'Testing model training and predictions...'
45         sh '''
46             . venv/bin/activate
47             pytest tests/test_model.py -v --tb=short
48         '''
49     }
50 }
51
52 stage('Test API') {
53     steps {
54         echo 'Testing FastAPI application...'
55         sh '''
56             . venv/bin/activate
57             pytest tests/test_app.py -v --tb=short
58         '''
59     }
60 }
61
62 stage('Build Docker Image') {
63     steps {
64         echo 'Building Docker image...'
65         script {
```

```

66         docker.build("${DOCKER_IMAGE}:${DOCKER_TAG}")
67         docker.build("${DOCKER_IMAGE}:latest")
68     }
69 }
70 }
71
72 stage('Push to Docker Hub') {
73     steps {
74         echo 'Pushing to Docker Hub...'
75         script {
76             docker.withRegistry('https://registry.hub.docker.com', "${
77                 DOCKER_CREDENTIALS_ID}") {
78                 docker.image("${DOCKER_IMAGE}:${DOCKER_TAG}").push()
79                 docker.image("${DOCKER_IMAGE}:latest").push()
80             }
81         }
82     }
83
84     stage('Cleanup') {
85         steps {
86             echo 'Cleaning up...'
87             sh '''
88                 docker rmi ${DOCKER_IMAGE}:${DOCKER_TAG} || true
89                 docker rmi ${DOCKER_IMAGE}:latest || true
90                 rm -rf venv
91             '''
92         }
93     }
94 }
95
96 post {
97     success {
98         echo 'Pipeline completed successfully!'
99         echo "Docker image pushed: ${DOCKER_IMAGE}:${DOCKER_TAG}"
100     }
101     failure {
102         echo 'Pipeline failed!'
103     }
104     always {
105         echo 'Cleaning workspace...'
106         cleanWs()
107     }
108 }
109 }

```

Code trên định nghĩa quy trình CI/CD (Continuous Integration/Continuous Deployment). Đây là dạng Declarative Pipeline, giúp quy trình rõ ràng và dễ đọc. Với các cấu trúc chính như:

- **agent any:** Pipeline có thể chạy trên bất kỳ node (máy trạm) nào khả dụng của Jenkins.
- **environment:** Khai báo các biến môi trường dùng chung (Tên Docker Image, Credentials ID để đăng nhập Docker Hub).

Tiếp theo chúng ta sẽ đi sâu thêm về các Stages đã định nghĩa:

- **Stage: Checkout** - Lấy mã nguồn mới nhất từ kho chứa (Git) về workspace của Jenkins để bắt đầu xử lý.
- **Stage: Setup Python Environment** - Tạo một môi trường ảo (venv) sạch sẽ, để tránh xung đột thư viện giữa các lần chạy (build) khác nhau hoặc với hệ thống chính. Nó cài đặt các thư viện từ requirements.txt.
- **Stage: Train Model** - Kích hoạt file src/train_model.py.
- **Stage: Test Mode** - Chạy tests/test_model.py.
- **Stage: Test API** - Chạy tests/test_app.py.
- **Stage: Build Docker Image** - Dùng Dockerfile để đóng gói toàn bộ ứng dụng (code + model đã train) thành một Docker Image.
- **Stage: Push to Docker Hub** - Đăng nhập vào Docker Hub (dùng credentials đã lưu trong Jenkins) và đẩy image lên. Sau bước này, bạn có thể pull image về bất cứ server nào để chạy.
- **Stage: Cleanup** - Xóa image trên máy Jenkins và thư mục venv để giải phóng dung lượng đĩa cứng sau khi hoàn tất.

Sau khi xây dựng xong các files cần thiết lúc này chúng ta sẽ bắt đầu chạy Jenkins container như sau:

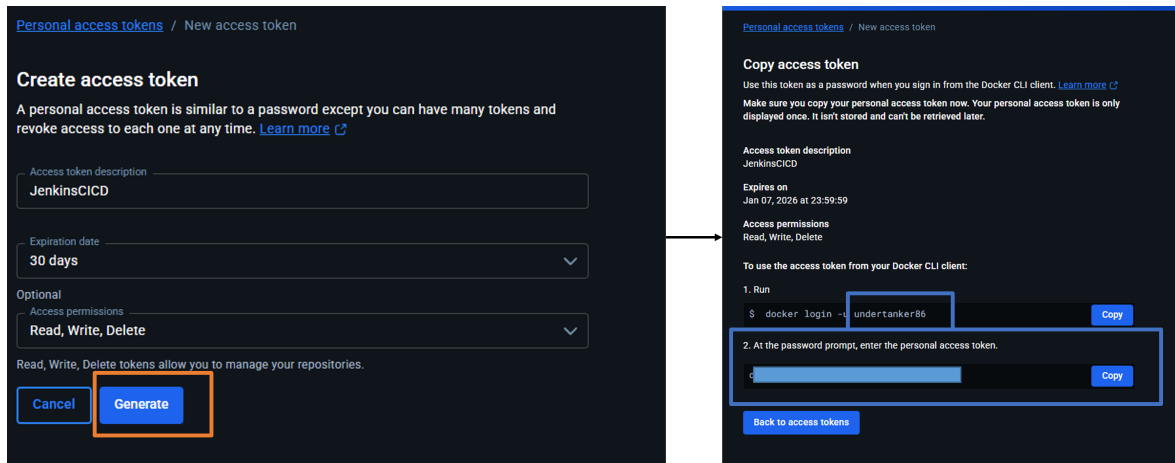
```
run
1 # Move to jenkins folder
2 cd jenkins
3
4 # Run Jenkins
5 docker-compose up -d
6
7 # Check container running
8 docker ps
```

Về cấu hình đăng nhập Jenkins, mọi người có thể tham khảo thêm tại Github (Triển khai với Jenkins) trong mục **Source Code**.

Bên cạnh đó chúng ta cũng sẽ cần phải tạo một Github repository để có thể đẩy code và lưu trữ code.

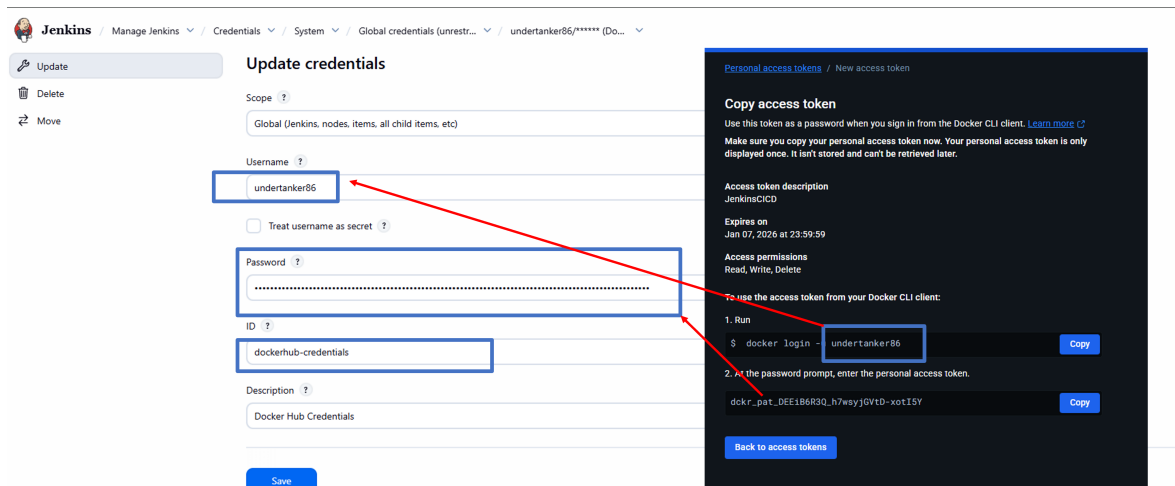
III.1.5. Bước 5: Thiết lập Docker Credential

Ở đây chúng ta sẽ cần phải tạo tài khoản và đăng nhập Dockerhub (<https://hub.docker.com/>) để có thể tạo và lấy "access tokens"(Hình số 8) từ đó Jenkins sẽ tìm credentials với ID chính xác để sử dụng đẩy Docker Image lên Dockerhub (public registry).



Hình 8: Tạo access token trên Dockerhub.

Sau khi đã có "access token" của Dockerhub giờ chúng ta sẽ tiến hành thêm thông tin vào trong Jenkins credentials như hình số 9, lưu ý một điều ở mục ID cần điền chuẩn tên "dockerhub-credentials".



Hình 9: Cấu hình Docker credential

III.1.6. Bước 6: Thiết lập các plugins và cấu hình pipeline

Ở bước này chúng ta cần tải các plugin chính sau: Git plugin, Github plugin, Docker plugin, Docker Pipeline, Pipeline.

Sau khi đã có các plugins cần thiết tiếp theo chúng ta sẽ tiến hành cấu hình pipeline: Từ Dashboard sẽ click mục "New Item" sau đó điền thông tin "item name" và chọn "item type" là Pipeline (Cấu hình như hình số 10).

New Item

Enter an item name

iris-ml-cicd

Select an item type

 **Freestyle project**
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.

 **Pipeline**
Orchestrates long-running activities that can span multiple build agents. Suitable for building pipelines (formerly known as workflows) and/or organizing complex activities that do not easily fit in free-style job type.

OK

Hình 10: Thêm mới "Item" trong Jenkins.

Ở mục "Triggers" chúng ta sẽ chọn trigger là "GitHub hook trigger for GITScm polling" nghĩa là sẽ nhận trigger luồng CI/CD từ Github hook. Tiếp theo sẽ chọn Definition là Pipeline script from SCM và chọn SCM là Git và điền Github repository URL đã tạo ở bước trên (Lưu ý nên để repository là chế độ public để tránh phải config thêm Credential).

Jenkins / iris-ml-cicd / Configuration

Configure

General

Triggers

Pipeline

Advanced

☐ Throttle builds

Triggers

Set up automated actions that start your build based on specific events, like code changes or scheduled times.

☐ Build after other projects are built

☐ Build periodically

☐ Generic Webhook Trigger

☒ GitHub hook trigger for GITScm polling

☐ Poll SCM

Pipeline

Define your Pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script from SCM

SCM

Git

Repositories

Repository URL

https://github.com/undertanker86/iris-cicd-jenkins

Credentials

- none -

+ Add

Save Apply

Hình 11: Cấu hình pipeline với Github hook.

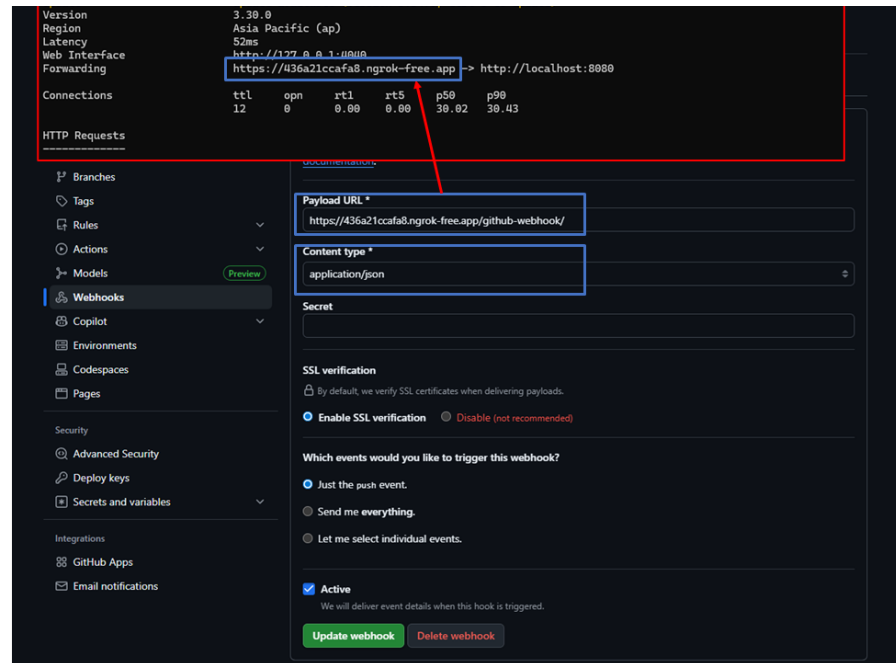
Tiếp theo, hệ thống cần xác định nhánh mã nguồn chịu trách nhiệm kích hoạt pipeline, cụ thể là nhánh master. Cuối cùng, tại trường cấu hình "Script Path", đường dẫn cần được tham chiếu trực tiếp đến tệp Jenkinsfile nhằm định nghĩa các bước vận hành tự động (Hình số 12).

Hình 12: Cấu hình nhánh và script path.

III.1.7. Bước 7: Thiết lập Github hook

Bước này yêu cầu thiết lập cơ chế GitHub Webhook nhằm tự động kích hoạt quy trình (trigger) mỗi khi có sự kiện đẩy mã nguồn (push event) lên kho lưu trữ. Phương pháp cấu hình URL cho Webhook phụ thuộc vào kiến trúc mạng của máy chủ Jenkins. Đối với máy chủ sở hữu địa chỉ IP công khai (Public IP), đường dẫn kết nối có thể được thiết lập trực tiếp. Tuy nhiên, trong phạm vi triển khai của bài thực hành, do Jenkins vận hành trên môi trường cục bộ (localhost) và không được định tuyến công khai, giải pháp sử dụng công cụ ngrok được áp dụng. Ngrok đóng vai trò thiết lập đường hầm (tunneling) an toàn, khởi tạo một URL công khai (public URL) để đáp ứng yêu cầu kết nối từ phía GitHub.

Sau khi hoàn tất cài đặt ngrok, chúng ta sẽ thực thi câu lệnh "ngrok http 8080" nhằm thiết lập đường dẫn truy cập công khai (public URL) cho dịch vụ Jenkins đang chạy nội bộ. Tiếp theo, tại giao diện cấu hình "Settings/Webhooks" của Github repository, cài đặt một Webhook bằng cách nhập địa chỉ URL vừa sinh ra vào trường "Payload URL", đồng thời thiết lập trường "Content type" là application/json. Chi tiết các bước cấu hình được minh họa cụ thể tại hình 13.



Hình 13: Thiết lập Github webhook.

III.1.8. Bước 8: Thực hiện push code và tiến hành trigger

Sau khi đã cấu hình xong các bước trên, bây giờ chúng ta chỉ sẽ thực hiện push code lên Github repository đã được tạo kích hoạt luồng chạy CI/CD và thu được kết quả như sau:

```
Training ML model...
[Pipeline] sh
+ . venv/bin/activate
+ deactivate nondestructive
+ [ -n ]
+ [ -n ]
+ hash -r
+ [ -n ]
+ unset VIRTUAL_ENV
+ unset VIRTUAL_ENV_PROMPT
+ [ ! nondestructive = nondestructive ]
+ uname
+ export VIRTUAL_ENV=/var/jenkins_home/workspace/iris-ml-cicd/venv
+ _OLD_VIRTUAL_PATH=/opt/java/openjdk/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
+ PATH=/var/jenkins_home/workspace/iris-ml-cicd/venv/bin:/opt/java/openjdk/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
+ export PATH
+ VIRTUAL_ENV_PROMPT=venv
+ export VIRTUAL_ENV_PROMPT
+ [ -n ]
+ [ -z ]
+ _OLD_VIRTUAL_PS1=#
+ PS1=(venv) #
+ export PS1
+ hash -r
+ cd src
+ python train_model.py
Model accuracy: 1.00
Model saved to models/iris_model.pkl
```

Hình 14: Log của quá trình huấn luyện mô hình.

```
+ pytest tests/test_model.py -v --tb=short
===== test session starts =====
platform linux -- Python 3.13.5, pytest-8.3.0, pluggy-1.6.0 --
/var/jenkins_home/workspace/iris-ml-cicd/venv/bin/python3
cachedir: .pytest_cache
rootdir: /var/jenkins_home/workspace/iris-ml-cicd
plugins: anyio-4.12.0
collecting ... collected 6 items

tests/test_model.py::TestModelTraining::test_train_model_success PASSED [ 16%]
tests/test_model.py::TestModelTraining::test_model_accuracy_threshold PASSED [ 33%]
tests/test_model.py::TestModelTraining::test_model_file_created PASSED [ 50%]
tests/test_model.py::TestModelTraining::test_model_can_be_loaded PASSED [ 66%]
tests/test_model.py::TestModelTraining::test_loaded_model_can_predict PASSED [ 83%]
tests/test_model.py::TestModelTraining::test_model_parameters PASSED [100%]

===== 6 passed in 0.99s =====
```

Hình 15: Log của quá trình test mô hình.

```
Testing FastAPI application...
[Pipeline] sh
+ . venv/bin/activate
+ deactivate nondestructive
+ [ -n ]
+ [ -n ]
+ hash -r
+ [ -n ]
+ unset VIRTUAL_ENV
+ unset VIRTUAL_ENV_PROMPT
+ [ ! nondestructive = nondestructive ]
+ uname
+ export VIRTUAL_ENV=/var/jenkins_home/workspace/iris-ml-cicd/venv
+ _OLD_VIRTUAL_PATH=/opt/java/openjdk/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
+ PATH=/var/jenkins_home/workspace/iris-ml-cicd/venv/bin:/opt/java/openjdk/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
+ export PATH
+ VIRTUAL_ENV_PROMPT=venv
+ export VIRTUAL_ENV_PROMPT
+ [ -n ]
+ [ -z ]
+ _OLD_VIRTUAL_PS1=#
+ PS1=(venv) #
+ export PS1
+ hash -r
+ pytest tests/test_app.py -v --tb=short
===== test session starts =====
platform linux -- Python 3.13.5, pytest-8.3.0, pluggy-1.6.0 -- /var/jenkins_home/workspace/iris-ml-cicd/venv/bin/python3
cachedir: .pytest_cache
rootdir: /var/jenkins_home/workspace/iris-ml-cicd
plugins: anyio-4.12.0
collecting ... collected 4 items

tests/test_app.py::test_root PASSED [ 25%]
tests/test_app.py::test_health PASSED [ 50%]
tests/test_app.py::test_predict PASSED [ 75%]
tests/test_app.py::test_predict_invalid_input PASSED [100%]
```

Hình 16: Log của quá trình test API.


```

#4 [1/7] FROM docker.io/library/python:3.10-slim@sha256:c299e10e007017113f9a1f109dd05e7e634fa94589b056e0e87bb22b2b382a2
#4 resolve docker.io/library/python:3.10-slim@sha256:c299e10e007017113f9a1f109dd05e7e634fa94589b056e0e87bb22b2b382a2 0.0s done
#4 DONE 0.0s

#5 [internal] load build context
#5 transferring context: 128B done
#5 DONE 0.0s

#6 [5/7] COPY src/train_model.py .
#6 CACHED

#7 [6/7] COPY src/app.py .
#7 CACHED

#8 [2/7] WORKDIR /app
#8 CACHED

#9 [3/7] COPY requirements.txt .
#9 CACHED

#10 [4/7] RUN pip install --no-cache-dir -r requirements.txt
#10 CACHED

#11 [7/7] RUN python train_model.py
#11 CACHED

#12 exporting to image
#12 exporting layers done
#12 exporting manifest sha256:83b6668afa01ff18a51636621413b1d4a9d53f0c0f930f217e4106b3dcf72746 0.0s done
#12 exporting config sha256:9029410f428371773875511e8dc9476559f0f8110c8c32b1dc29a6365a9122f0 0.0s done
#12 exporting attestation manifest sha256:6fd23f98e0b6ce63a9fea9cfedc436d156eb22241779c7f7bc3e5d1b5220f863 0.0s done
#12 exporting manifest list sha256:e83b718e45c9bb10b9b176068cf0bfbdb6b3234fe550c630798336a2adb032 0.0s done
#12 exporting manifest list sha256:e83b718e45c9bb10b9b176068cf0bfbdb6b3234fe550c630798336a2adb032 0.0s done
#12 naming to docker.io/undertanker86/iris-ml-api:latest done
#12 unpacking to docker.io/undertanker86/iris-ml-api:latest 0.0s done
#12 DONE 0.1s

```

Hình 17: Log của quá trình đóng gói Docker Image.

```

+ docker rmi undertanker86/iris-ml-api:5
Untagged: undertanker86/iris-ml-api:5
+ docker rmi undertanker86/iris-ml-api:latest
Untagged: undertanker86/iris-ml-api:latest
+ rm -rf venv
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Cleaning workspace...
[Pipeline] cleanWs
[WS-CLEANUP] Deleting project workspace...
[WS-CLEANUP] Deferred wipeout is used...
[WS-CLEANUP] done
[Pipeline] echo
Pipeline completed successfully!
[Pipeline] echo
Docker image pushed: undertanker86/iris-ml-api:5
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS

```

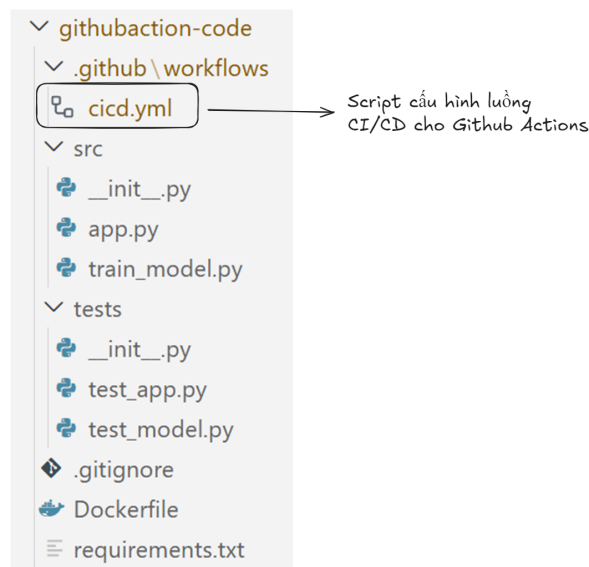
Tag	OS	Type	Pulled	Pushed
latest		Image	less than 1 day	about 7 hours
5		Image	less than 1 day	about 7 hours

Hình 18: Kết quả của quá trình push Docker Image lên Dockerhub.

III.2. Triển khai với Github Actions

Đối với phương án triển khai này, ở các bước đầu chúng ta sẽ làm tương tự với bước 1, 2 và 3 đã được trình bày trong phần **Triển khai với Jenkins**. Sự khác biệt cốt lõi nằm ở cơ chế vận

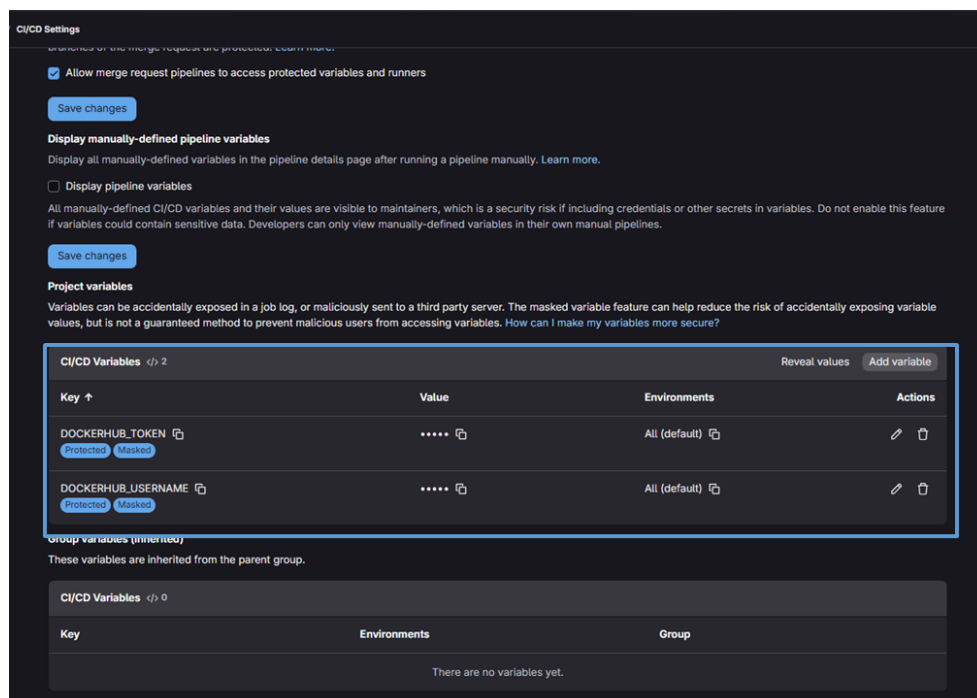
hành: thay vì sử dụng máy chủ Jenkins, hệ thống sẽ chuyển sang cấu hình Github Actions để điều phối luồng CI/CD. Cấu trúc code sẽ được định nghĩa cụ thể như sau:



Hình 19: Cấu trúc code thực hành Github Actions.

III.2.1. Bước 4: Cấu hình CI/CD pipeline và thiết lập Docker Credential

Ở bước này chúng ta sẽ cần truy cập vào Github (<https://github.com/>) và tạo một repository. Tiếp theo truy cập vào mục "Actions" chọn mục "set up a workflow yourself" và giao diện sẽ được hiển thị như sau:



Hình 20: Giao diện tạo file cấu hình CI/CD trong Github Actions.

Sau đó sẽ lựa chọn đổi tên "main.yml" thành "cicd.yml" (Tùy chọn) và chọn "Commit change". Khi tạo và commit thành công "cicd.yml" lúc này chúng ta sẽ chỉ cần pull request về và thực hiện điều chỉnh code cicd.yml như sau:

cicd.yml

```
1 name: CI/CD Pipeline
2
3 on:
4   push:
5     branches:
6       - main
7       - develop
8   workflow_dispatch:
9
10 env:
11   DOCKER_IMAGE: your-docker-name/iris-ml-api
12   PYTHON_VERSION: '3.13'
13
14 jobs:
15   build-and-deploy:
16     runs-on: ubuntu-latest
17
18     steps:
19       - name: Checkout Code
20         uses: actions/checkout@v4
21
22       - name: Set up Python
23         uses: actions/setup-python@v5
24         with:
25           python-version: ${ env.PYTHON_VERSION }
26           cache: 'pip'
27
28       - name: Install Dependencies
29         run: |
30           python -m pip install --upgrade pip
31           pip install -r requirements.txt
32
33       - name: Train ML Model
34         run: |
35           cd src
36           python train_model.py
37           cd ..
38
39       - name: Test Model
40         run: |
41           pytest tests/test_model.py -v --tb=short
42
43       - name: Test API
44         run: |
45           pytest tests/test_app.py -v --tb=short
46
47       - name: Set up Docker Buildx
48         uses: docker/setup-buildx-action@v3
```

```

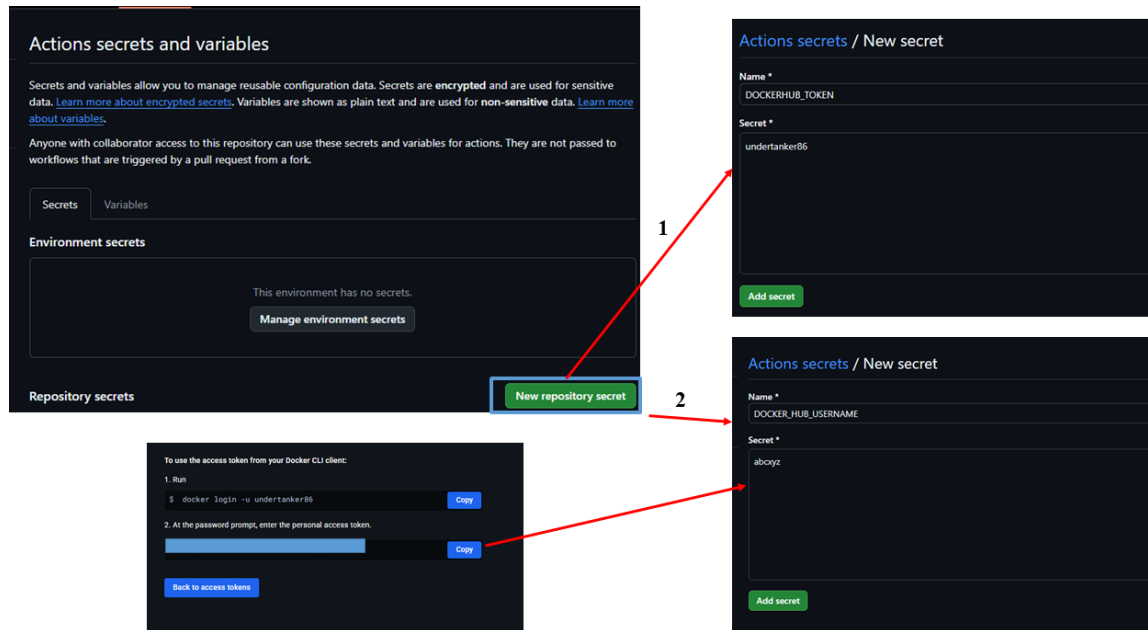
49
50   - name: Login to Docker Hub
51     uses: docker/login-action@v3
52     with:
53       username: ${ secrets.DOCKERHUB_USERNAME }}
54       password: ${ secrets.DOCKERHUB_TOKEN }}
55
56   - name: Build and Push Docker Image
57     uses: docker/build-push-action@v5
58     with:
59       context: .
60       push: true
61       tags: |
62         ${ env.DOCKER_IMAGE }}:${ github.run_number }}
63         ${ env.DOCKER_IMAGE }}:latest
64       cache-from: type=gha
65       cache-to: type=gha,mode=max
66
67   - name: Image digest
68     run: echo "Docker image pushed successfully with tag ${ github.run_number }}"
69
70   - name: Cleanup
71     if: always()
72     run: |
73       echo "Cleaning up temporary files..."
74       rm -rf __pycache__
75       rm -rf .pytest_cache

```

Đoạn code trên là cấu hình một pipeline trong Github Actions cụ thể như sau:

- **name:** hiển thị tên của workflow.
- **on:** dùng để định nghĩa khi nào workflow chạy ở đây sẽ là khi push request vào nhánh main hoặc develop.
- Ở đây sẽ có thêm các biến môi trường như DOCKER_IMAGE - Tên Docker image sẽ được push lên Docker Hub (username/repository), và PYTHON_VERSION: Phiên bản Python để chạy test và train model.
- **build-and-deploy:** Tên job duy nhất trong workflow này, và **runs-on: ubuntu-latest:** Chạy trên máy ảo Ubuntu mới nhất của GitHub.
- Cuối cùng sẽ là các steps thực thi với các nội dung chính như: Checkout Code -> Setup Python -> Cài đặt Dependencies -> Train ML Model -> Test Model và API -> Setup Docker Buildx -> Login Docker Hub -> Build và Push Docker Image -> Thông báo và Cleanup.

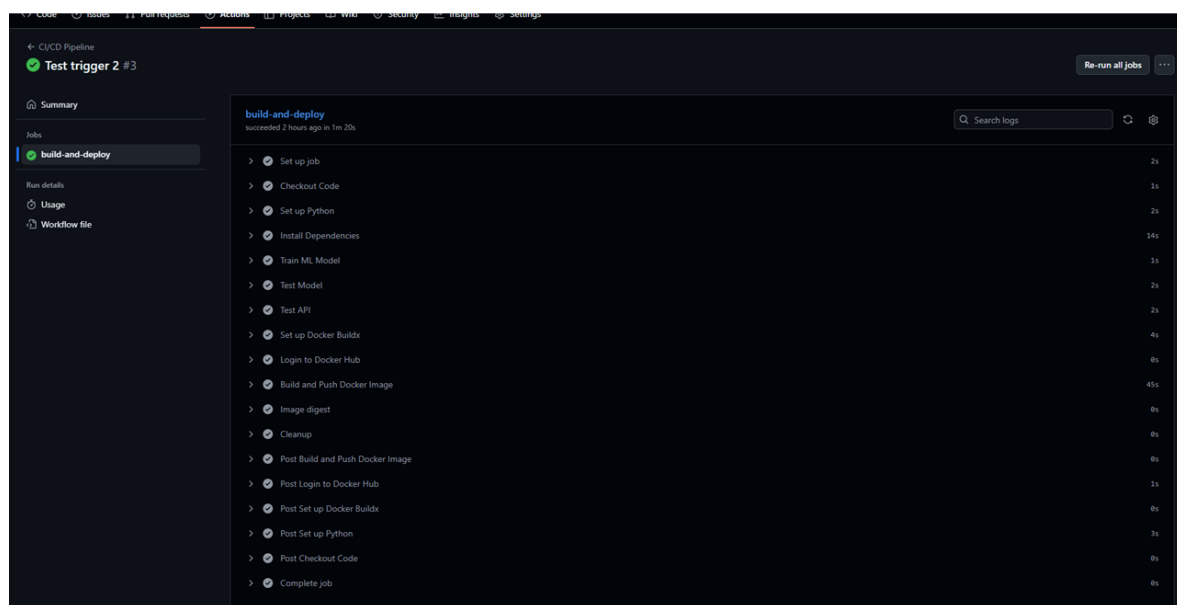
Sau khi đã cấu hình thành công pipeline CI/CD chúng ta sẽ cần phải thiết lập thêm Docker Credential để có thể đẩy Docker Image lên Dockerhub. Về cách lấy "access token" từ Dockerhub mọi người có thể tham khảo ở mục **Triển khai với Jenkins/ Bước 5: Thiết lập Docker Credential**. Sau khi đã có "access token" lúc này chúng ta sẽ truy cập vào mục "Settings -> Secrets and Variables -> Actions" từ đó thêm hai thông tin secret như Hình số 21:



Hình 21: Thêm thông tin secrets: Dockerhub username và Dockerhub access token.

III.2.2. Bước 5: Thực hiện push code và tiến hành trigger luồng CI/CD

Sau khi đã làm xong các bước trên, lúc này chúng ta sẽ chỉ cần push code lên repository để kích hoạt luồng chạy CI/CD, để xem được pipeline chúng ta truy cập vào mục Actions và cuối cùng sẽ được kết quả như sau:

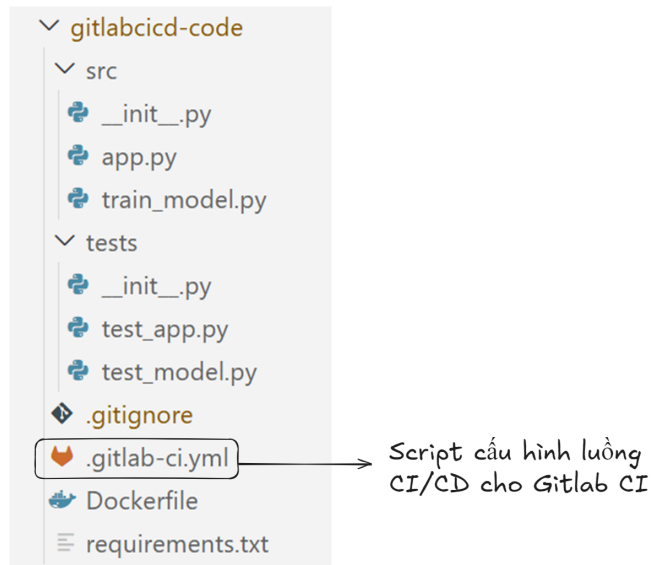


Hình 22: Kết quả luồng CI/CD với Github Actions

III.3. Triển khai với Gitlab CI

Lưu ý: Nhằm đảm bảo an toàn hệ thống và ngăn chặn lạm dụng tài nguyên, nền tảng GitLab áp dụng chính sách xác thực danh tính nghiêm ngặt. Để kích hoạt luồng CI/CD, tài khoản người dùng bắt buộc phải được xác minh thông qua số điện thoại và thông tin thanh toán hợp lệ.

Việc triển khai ban đầu sẽ tương tự như ở bên Github Actions, chúng ta vẫn sẽ giữ nguyên cấu hình các bước 1,2,3 và chỉ điều chỉnh chuyển sang cấu hình Gitlab CI để điều phối luồng CI/CD.



Hình 23: Cấu trúc code thực hành Gitlab CI.

III.3.1. Bước 4: Cấu hình .gitlab-ci.yml và thiết lập Docker Credentials

```
.gitlab-ci.yml
1 stages:
2   - setup
3   - train
4   - test
5   - build
6   - deploy
7   - cleanup
8
9 variables:
10  DOCKER_IMAGE: "your-docker-name/iris-ml-api-gitlab"
11  DOCKER_DRIVER: overlay2
12  PYTHON_VERSION: "3.13"
13  PIP_CACHE_DIR: "$CI_PROJECT_DIR/.cache/pip"
14
15  # Cache to speed up the build process
16  cache:
17    paths:
18      - .cache/pip
```

```

19     - venv/
20
21 # Template to reuse for Python jobs
22 .python_template: &python_setup
23 image: python:${PYTHON_VERSION}-slim
24 before_script:
25     - python -m venv venv
26     - source venv/bin/activate
27     - pip install --upgrade pip
28     - pip install -r requirements.txt
29
30 # Stage 1: Setup and install dependencies
31 setup_dependencies:
32     <<: *python_setup
33 stage: setup
34 script:
35     - echo "Installing dependencies..."
36     - pip list
37 artifacts:
38     paths:
39         - venv/
40     expire_in: 1 hour
41
42 # Stage 2: Train ML Model
43 train_model:
44     <<: *python_setup
45 stage: train
46 dependencies:
47     - setup_dependencies
48 script:
49     - echo "Training ML model..."
50     - cd src
51     - python train_model.py
52     - ls -la models/*.pkl
53     - cd ..
54 artifacts:
55     paths:
56         - src/models/iris_model.pkl
57     expire_in: 1 hour
58
59 # Stage 3: Test Model
60 test_model:
61     <<: *python_setup
62 stage: test
63 dependencies:
64     - setup_dependencies
65     - train_model
66 script:
67     - echo "Testing ML model..."
68     - pytest tests/test_model.py -v --tb=short
69 coverage: '/(?i)total.*? (100(?:\.\d+)?)%|[1-9]?[d](?:\.\d+)?\%)/'
70
71 # Stage 3: Test API
72 test_api:

```

```

73 <<: *python_setup
74 stage: test
75 dependencies:
76   - setup_dependencies
77   - train_model
78 script:
79   - echo "Testing FastAPI application..."
80   - pytest tests/test_app.py -v --tb=short
81
82 # Stage 4: Build Docker Image
83 build_docker_image:
84   stage: build
85   image: docker:24-dind
86   services:
87     - docker:24-dind
88   dependencies:
89     - train_model
90   before_script:
91     - docker info
92   script:
93     - echo "Building Docker image..."
94     - docker build -t ${DOCKER_IMAGE}:${CI_PIPELINE_IID} .
95     - docker build -t ${DOCKER_IMAGE}:latest .
96     - docker images
97   only:
98     - main
99     - develop
100
101 # Stage 5: Push to Docker Hub
102 push_to_dockerhub:
103   stage: deploy
104   image: docker:24-dind
105   services:
106     - docker:24-dind
107   dependencies:
108     - build_docker_image
109   before_script:
110     - echo "Logging into Docker Hub..."
111     - echo $DOCKERHUB_TOKEN | docker login -u $DOCKERHUB_USERNAME --password-stdin
112   script:
113     - echo "Pushing Docker image to Docker Hub..."
114     - docker build -t ${DOCKER_IMAGE}:${CI_PIPELINE_IID} .
115     - docker build -t ${DOCKER_IMAGE}:latest .
116     - docker push ${DOCKER_IMAGE}:${CI_PIPELINE_IID}
117     - docker push ${DOCKER_IMAGE}:latest
118     - echo "Docker image pushed successfully with tag ${CI_PIPELINE_IID}"
119   after_script:
120     - docker logout
121   only:
122     - main
123     - develop
124   when: on_success
125
126 # Stage 6: Cleanup

```



```

27 cleanup_workspace:
28   stage: cleanup
29   image: python:${PYTHON_VERSION}-slim
30   script:
31     - echo "Cleaning up temporary files..."
32     - rm -rf __pycache__
33     - rm -rf .pytest_cache
34     - rm -rf venv
35     - echo "Cleanup completed!"
36   when: always
37
38 # Job to run when there is a merge request
39 test_merge_request:
40   <<: *python_setup
41   stage: test
42   script:
43     - echo "Running tests for merge request..."
44     - cd src
45     - python train_model.py
46     - cd ..
47     - pytest tests/ -v --tb=short
48   only:
49     - merge_requests

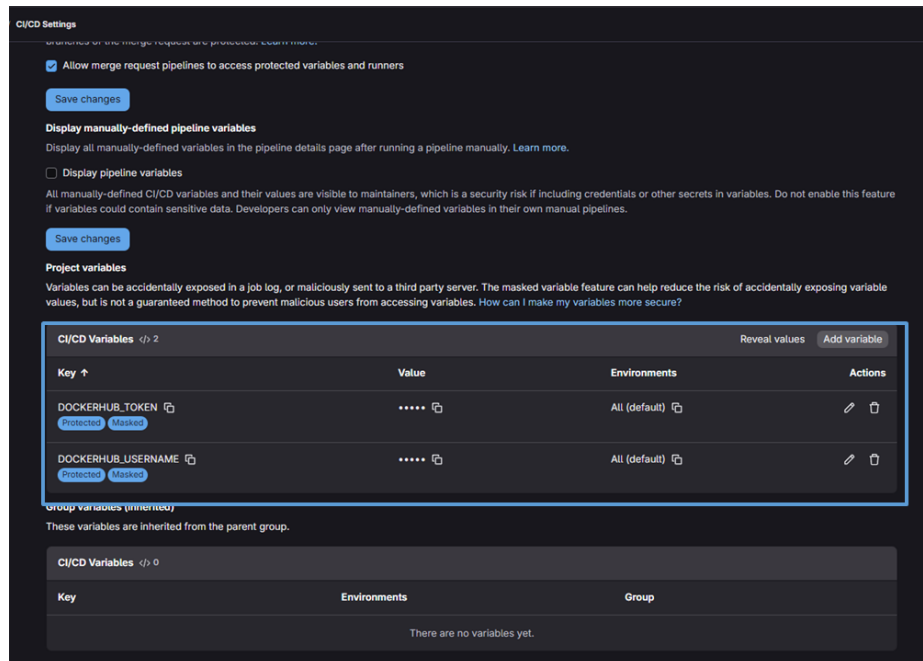
```

File .gitlab-ci.yml định nghĩa một pipeline CI/CD tự động hóa cho ứng dụng Machine Learning API. Pipeline được thiết kế theo kiến trúc phân tầng với 6 stages tuần tự: setup, train, test, build, deploy, và cleanup. Mỗi stage đảm nhiệm một trách nhiệm cụ thể trong quy trình từ cài đặt môi trường đến triển khai production. Các jobs trong cùng stage có thể chạy song song để tối ưu thời gian thực thi. Dưới đây sẽ phân tích kỹ hơn các components có trong code:

- **stages:** Phần này định nghĩa 6 giai đoạn pipeline: setup, train, test, build, deploy và cleanup, ngoài ra luồng chạy theo thứ tự và stage sau chỉ chạy khi stage trước thành công.
- **variables:** Có tổng cộng 4 biến môi trường bao gồm: DOCKER_IMAGE - Tên Docker image sẽ push lên Docker Hub, DOCKER_DRIVER - Driver cho Docker (overlay2 = hiệu năng tốt), PYTHON_VERSION - Phiên bản Python để chạy và cuối cùng PIP_CACHE_DIR - Thư mục lưu cache pip để tăng tốc.
- **cache:** đây là phần bổ sung với mục tiêu lưu thư mục .cache/pip và venv/ giữa các lần chạy pipeline giúp tăng tốc độ chạy pipeline.
- **Template .python_template:** Sử dụng YAML anchors (&python_setup) để định nghĩa configuration chung cho các Python jobs. Template này bao gồm Docker image specification, virtual environment setup, và dependencies installation. Các jobs sử dụng «: *python_setup để kế thừa configuration. Pattern này đặc biệt hữu ích khi cần update Python version hoặc installation steps.
- **Pipeline thực thi theo sequence:** Setup tạo venv và cài dependencies, lưu thành artifacts để reuse. Train stage huấn luyện ML model với Iris dataset, output là iris_model.pkl được pass qua artifacts. Test stage chạy song song test_model và test_api, verify model accuracy

và API functionality. Build stage containerize ứng dụng với Docker-in-Docker, tạo images với tags versioned và latest. Deploy stage authenticate và push images lên Docker Hub registry. Cleanup stage dọn dẹp temporary files để giải phóng resources.

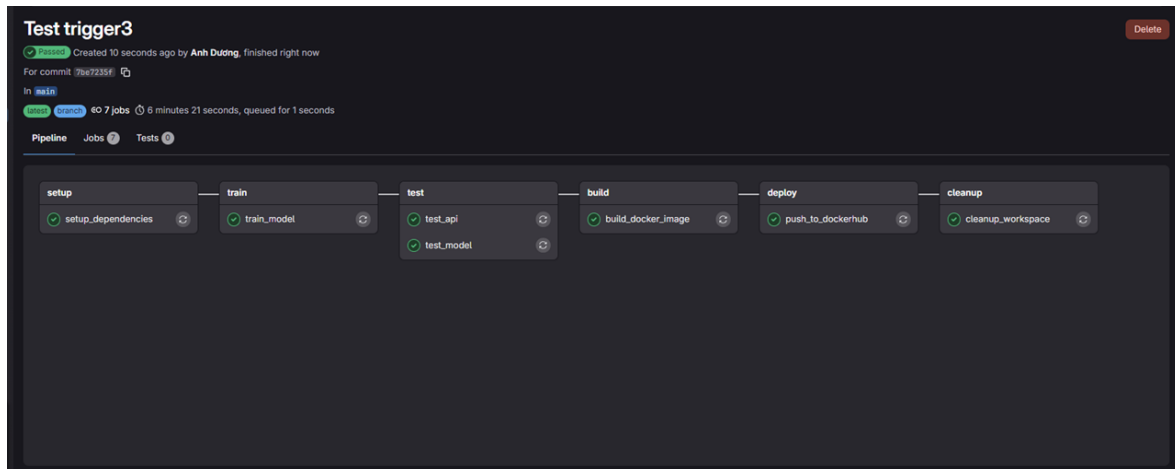
Tiếp theo chúng ta sẽ tạo một project với Gitlab (Có thể hiểu là tạo repository) để có thể tiến hành tạo request tiến hành trigger luồng CI/CD. Ngoài ra phần cấu hình Docker Credential để thực hiện đẩy Docker Image lên Dockerhub, chúng ta sẽ truy cập vào mục "Settings -> CI/CD" và thêm hai "variable" như Hình số 24.



Hình 24: Thiết lập Docker credential cho Gitlab CI.

III.3.2. Bước 5: Thực hiện push code và tiến hành trigger luồng CI/CD

Bước cuối cùng, chúng ta sẽ thực hiện push request lên Gitlab repository để tiến hành kích hoạt luồng CI/CD và thu được kết quả luồng chạy như sau:



Hình 25: Kết quả luồng chạy CI/CD với Gitlab CI.

IV. Kết luận

Tổng kết lại, bài viết đã hệ thống hóa nền tảng lý thuyết về CI/CD và làm rõ kiến trúc vận hành của ba công cụ tiêu chuẩn là Jenkins, GitHub Actions và GitLab CI. Ngoài ra, thông qua các cách triển khai luồng CI/CD thực tế với ba công cụ trên, bài viết đã giúp xây dựng một quy trình phát triển phần mềm chuẩn hóa, tạo tiền đề vững chắc để phát triển tiếp các hệ thống AI với quy mô lớn hơn, nơi việc tích hợp và chuyển giao sản phẩm được thực hiện tự động, chính xác và giảm thiểu tối đa rủi ro vận hành.

V. Tài liệu tham khảo

- <https://docs.github.com/en/actions/get-started/understand-github-actions>
- <https://www.jenkins.io/doc/>
- <https://docs.gitlab.com/tutorials/>
- <https://www.tutorialspoint.com/jenkins/index.htm>

VI. Source code và Demo

- **Triển khai với Jenkins:** <https://github.com/undertanker86/iris-cicd-jenkins>.
- **Triển khai với Github Actions:** <https://github.com/undertanker86/iris-cicd-githubaction>.
- **Triển khai với Gitlab CI:** <https://gitlab.com/undertanker86-group/iris-cicd-gitlabci>.
- **Demo:** <https://cicd-playground.streamlit.app/>